Top 111 Interview Questions

Introduction to Machine Learning Interviews Book · MLIB

Machine Learning Interview Questions & Answers

Commonly Asked at Google, Microsoft, Meta, and X

I'll organize these by category to help you prepare systematically.

---

1. Foundational ML Concepts

Q1: What is the difference between supervised and unsupervised learning?

Answer: Supervised learning uses labeled data where the algorithm learns to map inputs to known outputs (e.g., classification, regression). Unsupervised learning works with unlabeled data to find hidden patterns or structures (e.g., clustering, dimensionality reduction). Semi-supervised learning combines both approaches.

Q2: Explain the bias-variance tradeoff.

Answer: Bias is the error from overly simplistic assumptions in the model, leading to underfitting. Variance is the error from sensitivity to small fluctuations in training data, leading to overfitting. The goal is to find the sweet spot that minimizes total error. High bias means the model is too simple; high variance means it's too complex and doesn't generalize well.

Q3: What is overfitting and how do you prevent it?

Answer: Overfitting occurs when a model learns the training data too well, including noise, and performs poorly on new data. Prevention techniques include:

- Cross-validation
- Regularization (L1, L2)
- Early stopping
- Dropout (for neural networks)
- Pruning (for decision trees)
- Getting more training data
- Feature selection
- Ensemble methods

Q4: Explain precision, recall, and F1-score.

Answer:

- **Precision** = TP/(TP+FP) - Of all positive predictions, how many were correct?
- **Recall** = TP/(TP+FN) - Of all actual positives, how many did we find?
- **F1-score** = 2 × (Precision × Recall)/(Precision + Recall) - Harmonic mean balancing both metrics

Use precision when false positives are costly; recall when false negatives are costly.

Q5: What is cross-validation and why is it important?

Answer: Cross-validation splits data into k folds, training on k-1 folds and validating on the remaining fold, rotating through all folds. It provides a more robust estimate of model performance than a single train-test split, helps detect overfitting, and maximizes use of limited data. K-fold CV is most common, with stratified k-fold maintaining class distributions.

---

2. Algorithms & Model Selection

Q6: Explain how decision trees work and their advantages/disadvantages.

Answer: Decision trees split data recursively based on features that maximize information gain or minimize impurity (Gini, entropy).

- **Advantages:** Interpretable, handles non-linear relationships, requires little preprocessing, handles mixed data types
- **Disadvantages:** Prone to overfitting, unstable (small data changes cause big tree changes), biased toward features with many levels

Q7: What is the difference between bagging and boosting?

Answer:

- **Bagging** (Bootstrap Aggregating): Trains models in parallel on random subsets with replacement, then averages predictions. Reduces variance. Example: Random Forest.
- **Boosting**: Trains models sequentially, with each model correcting errors of previous ones. Reduces bias. Examples: AdaBoost, Gradient Boosting, XGBoost.

Q8: Explain Random Forest and why it performs well.

Answer: Random Forest is an ensemble of decision trees trained on bootstrapped samples with random feature subsets at each split. It performs well because: it reduces overfitting through averaging, handles high-dimensional data, captures complex interactions, provides feature importance, and is robust to outliers. The randomness decorrelates trees, making the ensemble more effective.

Q9: How does logistic regression work?

Answer: Logistic regression models probability of a binary outcome using the logistic function: $P(y=1|x) = 1/(1 + e^{(-z)})$, where $z = \beta_0 + \beta_1 x_1 + \ldots + \beta_n x_n$. It uses maximum likelihood estimation to find coefficients. Despite the name, it's a classification algorithm. The output is a probability between 0 and 1, which can be thresholded for binary classification.

Q10: What is gradient descent and how does it work?

Answer: Gradient descent is an optimization algorithm that iteratively updates parameters to minimize a cost function. It calculates the gradient (partial derivatives) of the cost function with respect to parameters and moves in the opposite direction: $\theta = \theta - \alpha\nabla J(\theta)$, where α is the learning rate. Variants include batch gradient descent, stochastic gradient descent (SGD), and mini-batch gradient descent.

Q11: Explain SVM (Support Vector Machines).

Answer: SVM finds the optimal hyperplane that maximizes the margin between classes. Support vectors are the data points closest to the decision boundary. The kernel trick allows SVM to efficiently handle non-linear boundaries by implicitly mapping data to higher dimensions. Common kernels: linear, polynomial, RBF (Gaussian). SVMs work well in high-dimensional spaces but are computationally expensive for large datasets.

Q12: What is the kernel trick in SVM?

Answer: The kernel trick allows algorithms to operate in high-dimensional feature spaces without explicitly computing coordinates in that space. Instead of transforming data and computing dot products in the new space, kernels compute dot products in the original space that are equivalent to the transformed space. This makes non-linear classification computationally feasible.

---

3. Deep Learning

Q13: Explain backpropagation.

Answer: Backpropagation computes gradients of the loss function with respect to network weights using the chain rule. It propagates errors backward from output to input layers, calculating how much each weight contributed to the error. These gradients are then used to update weights via gradient descent. It's the foundation for training neural networks efficiently.

Q14: What is a convolutional neural network (CNN) and where is it used?

Answer: CNNs are designed for grid-like data (images, video). They use convolutional layers that apply filters to detect local patterns, pooling layers for dimensionality reduction, and fully connected layers for classification. Key features include parameter sharing and local connectivity. Used in: image classification, object detection, facial recognition, medical imaging, and autonomous vehicles.

Q15: Explain RNN and LSTM.

Answer:

- **RNN** (Recurrent Neural Network): Processes sequential data by maintaining hidden states. Suffers from vanishing/exploding gradient problems in long sequences.
- **LSTM** (Long Short-Term Memory): Special RNN with gates (forget, input, output) that control information flow, allowing it to learn long-term dependencies. Solves the vanishing gradient problem through its cell state mechanism.

Q16: What is dropout and why is it used?

Answer: Dropout randomly deactivates neurons during training with probability p (typically 0.2-0.5). This prevents co-adaptation of neurons, acts as regularization, and effectively trains an ensemble of sub-networks. During inference, all neurons are active but outputs are scaled by (1-p). It significantly reduces overfitting in deep neural networks.

Q17: Explain batch normalization.

Answer: Batch normalization normalizes inputs to each layer across the mini-batch, reducing internal covariate shift. It normalizes to mean 0 and variance 1, then applies learned scaling and shifting parameters. Benefits include faster training, higher learning rates, reduced sensitivity to initialization, and acts as regularization. Applied before or after activation functions.

Q18: What are activation functions and why are they needed?

Answer: Activation functions introduce non-linearity, enabling neural networks to learn complex patterns. Common ones:

- **ReLU**: f(x) = max(0, x) - Fast, avoids vanishing gradients, but can die
- **Sigmoid**: f(x) = 1/(1+e^-x) - Outputs [0,1], used in output layer for binary classification
- **Tanh**: f(x) = (e^x - e^-x)/(e^x + e^-x) - Outputs [-1,1], zero-centered
- **Leaky ReLU**: Prevents dying ReLU problem
- **Softmax**: Multi-class classification output layer

Q19: What is transfer learning?

Answer: Transfer learning uses knowledge from a pre-trained model on one task to solve a different but related task. Instead of training from scratch, you fine-tune a model trained on large datasets (like ImageNet). Benefits include reduced training time, less data requirements, and better performance. Common in computer vision and NLP (BERT, GPT models).

---

4. Natural Language Processing

Q20: Explain word embeddings (Word2Vec, GloVe).

Answer: Word embeddings represent words as dense vectors in continuous space where semantic similarity corresponds to vector proximity.

- **Word2Vec**: Uses skip-gram or CBOW to predict context words or target words. Captures semantic and syntactic relationships.
- **GloVe**: Constructs co-occurrence matrix and factorizes it to learn vectors that capture word-word co-occurrence statistics.

Q21: What is attention mechanism in transformers?

Answer: Attention allows models to focus on relevant parts of input when producing output. In transformers, self-attention computes attention scores between all pairs of positions: Attention(Q,K,V) = softmax(QK^T/√d_k)V. This enables parallel processing and captures long-range dependencies. Multi-head attention runs multiple attention mechanisms in parallel for richer representations.

Q22: Explain BERT and how it differs from GPT.

Answer:

- **BERT** (Bidirectional Encoder Representations from Transformers): Uses bidirectional transformers, pre-trained with masked language modeling and next sentence prediction. Encoder-only architecture, excels at understanding tasks.
- **GPT** (Generative Pre-trained Transformer): Unidirectional (left-to-right), decoder-only architecture, pre-trained with language modeling. Excels at generation tasks.

---

5. Model Evaluation & Metrics

Q23: How do you handle imbalanced datasets?

Answer: Techniques include:

- Resampling: oversample minority class (SMOTE) or undersample majority class
- Class weights: penalize misclassification of minority class more
- Anomaly detection approaches
- Ensemble methods
- Use appropriate metrics: F1-score, precision-recall, AUROC instead of accuracy
- Stratified sampling in cross-validation
- Generate synthetic samples

Q24: What is ROC curve and AUC?

Answer: ROC (Receiver Operating Characteristic) curve plots True Positive Rate vs False Positive Rate at various classification thresholds. AUC (Area Under Curve) measures the entire two-dimensional area underneath the ROC curve, ranging from 0 to 1. AUC of 0.5 indicates random classifier; 1.0 indicates perfect classifier. It's threshold-independent and useful for imbalanced datasets.

Q25: Explain confusion matrix.

Answer: A table showing true positives (TP), true negatives (TN), false positives (FP), and false negatives (FN). From it, we derive:

- Accuracy = (TP+TN)/(TP+TN+FP+FN)
- Precision = TP/(TP+FP)
- Recall/Sensitivity = TP/(TP+FN)
- Specificity = TN/(TN+FP)
- F1-Score = 2×Precision×Recall/(Precision+Recall)

Q26: When would you use Mean Absolute Error (MAE) vs Mean Squared Error (MSE)?

Answer:

- **MAE**: Average absolute difference between predictions and actual values. Less sensitive to outliers, all errors weighted equally, interpretable in original units.
- **MSE**: Average squared difference. Penalizes larger errors more heavily, differentiable everywhere (useful for optimization), sensitive to outliers. Use MAE when outliers should be treated equally; MSE when large errors are particularly undesirable.

---

6. Feature Engineering & Data Preprocessing

Q27: How do you handle missing data?

Answer: Strategies include:

- **Deletion**: Remove rows/columns with missing values (if small percentage)
- **Imputation**: Fill with mean/median/mode, forward/backward fill for time series, KNN imputation, predictive modeling
- **Indicator variable**: Add binary column indicating missingness
- Use algorithms that handle missing values natively (XGBoost)
- Consider if data is MCAR (Missing Completely At Random), MAR (Missing At Random), or MNAR (Missing Not At Random)

Q28: What is feature scaling and when is it needed?

Answer: Feature scaling transforms features to similar scales. Methods:

- **Normalization**: Scale to [0,1] range: (x - min)/(max - min)
- **Standardization**: Scale to mean 0, variance 1: (x - μ)/σ

Needed for: gradient descent-based algorithms, distance-based algorithms (KNN, SVM, K-means), neural networks, regularized models. Not needed for: tree-based algorithms.

Q29: Explain dimensionality reduction techniques.

Answer:

- **PCA** (Principal Component Analysis): Linear technique finding orthogonal directions of maximum variance
- **t-SNE**: Non-linear technique for visualization, preserves local structure
- **LDA** (Linear Discriminant Analysis): Supervised technique maximizing class separability
- **Autoencoders**: Neural network-based approach learning compressed representations Benefits: reduced computation, visualization, noise reduction, avoiding curse of dimensionality.

Q30: What is one-hot encoding and when would you use it?

Answer: One-hot encoding converts categorical variables into binary vectors where only one element is 1 and others are 0. Use for nominal (non-ordinal) categorical variables in algorithms that require numerical input (linear regression, neural networks, SVM). Alternative: label

encoding for ordinal variables or tree-based algorithms that can handle categorical data natively.

---

7. ML System Design & Production

Q31: How would you deploy a machine learning model in production?

Answer: Steps include:

1. Model serialization (pickle, joblib, ONNX)
2. Create API endpoint (Flask, FastAPI)
3. Containerization (Docker)
4. Orchestration (Kubernetes)
5. Set up monitoring and logging
6. A/B testing framework
7. Model versioning
8. CI/CD pipeline
9. Handle scaling and latency requirements
10. Set up retraining pipeline

Consider: serving latency, throughput, model updates, rollback strategy, data drift detection.

Q32: What is A/B testing in ML systems?

Answer: A/B testing compares two model versions by randomly assigning users to control (A) or treatment (B) groups. Measure key metrics (conversion rate, engagement, revenue) to determine statistical significance. Important considerations: sample size calculation, duration, stratification, multiple testing correction, interaction effects. Helps make data-driven decisions about model updates.

Q33: How do you monitor ML models in production?

Answer: Monitor:

- **Performance metrics**: accuracy, precision, recall, latency, throughput
- **Data quality**: missing values, distribution shifts, outliers
- **Data drift**: input feature distributions changing over time
- **Concept drift**: relationship between features and target changing
- **Model drift**: model performance degrading
- **System health**: CPU, memory, error rates

Set up alerts, dashboards, and automated retraining triggers.

Q34: What is model drift and how do you handle it?

Answer: Model drift occurs when model performance degrades over time due to:

- **Data drift**: Input distribution changes
- **Concept drift**: Target variable relationship changes
- **Upstream changes**: Data pipeline modifications

Handle by: regular performance monitoring, statistical tests for distribution changes, scheduled retraining, online learning, ensemble with recent models, canary deployments for new models.

Q35: How would you design a recommendation system?

Answer: Approaches:

- **Collaborative filtering**: User-based (similar users) or item-based (similar items). Use matrix factorization (SVD, ALS)
- **Content-based**: Recommend based on item features and user preferences
- **Hybrid**: Combine both approaches
- **Deep learning**: Neural collaborative filtering, two-tower models

Consider: cold start problem (new users/items), scalability, diversity, real-time updates, evaluation metrics (NDCG, MAP, recall@k).

---

8. Probability & Statistics

Q36: Explain the Central Limit Theorem.

Answer: The Central Limit Theorem states that the distribution of sample means approaches a normal distribution as sample size increases, regardless of the population's distribution (given finite variance). This is fundamental for hypothesis testing and confidence intervals. Typically $n \geq 30$ is considered sufficient for the approximation.

Q37: What is p-value and how do you interpret it?

Answer: P-value is the probability of obtaining test results at least as extreme as observed, assuming the null hypothesis is true. Low p-value (typically <0.05) suggests evidence against null hypothesis. Important: p-value is not the probability that null hypothesis is true, doesn't measure effect size, and can be misleading with large samples.

Q38: Explain Bayes' Theorem and its applications in ML.

Answer: Bayes' Theorem: $P(A|B) = P(B|A) \times P(A) / P(B)$

It updates probability of hypothesis given evidence. Applications:

- Naive Bayes classifier
- Bayesian networks
- Spam filtering
- Medical diagnosis

- A/B test interpretation
- Prior beliefs incorporation
- Posterior probability calculation

Q39: What is the difference between Type I and Type II errors?

Answer:

- **Type I error (α)**: False positive - rejecting true null hypothesis
- **Type II error (β)**: False negative - failing to reject false null hypothesis
- **Power (1-β)**: Probability of correctly rejecting false null hypothesis

Trade-off exists: reducing one typically increases the other. Choice depends on relative costs of each error type.

Q40: Explain maximum likelihood estimation (MLE).

Answer: MLE finds parameter values that maximize the likelihood of observing the given data. It assumes data is independent and identically distributed (i.i.d.). The likelihood function is the joint probability of observed data given parameters. In practice, we maximize log-likelihood for computational convenience. Used in: logistic regression, neural networks, Gaussian distributions.

---

9. Optimization & Mathematical Foundations

Q41: What is regularization? Explain L1 and L2.

Answer: Regularization adds penalty term to loss function to prevent overfitting:

- **L1 (Lasso)**: Adds sum of absolute values of coefficients. Performs feature selection by driving some coefficients to zero. Creates sparse models.
- **L2 (Ridge)**: Adds sum of squared coefficients. Shrinks coefficients but doesn't eliminate them. Preferred when all features are potentially relevant.
- **Elastic Net**: Combines both L1 and L2.

Q42: Explain the vanishing and exploding gradient problems.

Answer:

- **Vanishing gradients**: Gradients become extremely small during backpropagation, preventing early layers from learning. Common with sigmoid/tanh activations in deep networks.
- **Exploding gradients**: Gradients become extremely large, causing unstable learning and NaN values.

Solutions: ReLU activation, gradient clipping, batch normalization, skip connections (ResNet), LSTM/GRU for RNNs, careful weight initialization (Xavier, He).

Q43: What is learning rate and how do you choose it?

Answer: Learning rate (α) controls step size in gradient descent. Too high causes divergence; too low causes slow convergence or local minima. Selection strategies:

- Learning rate schedules: step decay, exponential decay, cosine annealing
- Adaptive methods: Adam, RMSprop, AdaGrad
- Learning rate finder: train briefly at increasing rates, plot loss
- Cyclical learning rates
- Warmup followed by decay

Typical starting values: 0.001-0.01.

Q44: Explain the difference between convex and non-convex optimization.

Answer:

- **Convex**: Single global minimum, any local minimum is global. Gradient descent guaranteed to find optimal solution. Examples: linear regression, logistic regression with convex loss.
- **Non-convex**: Multiple local minima, saddle points. No guarantee of finding global minimum. Examples: neural networks, most deep learning models.

Deep learning succeeds despite non-convexity through: good initialization, momentum, overparameterization, and the landscape having many good local minima.

---

10. Advanced Topics

Q45: Explain generative adversarial networks (GANs).

Answer: GANs consist of two neural networks competing:

- **Generator**: Creates fake samples from random noise
- **Discriminator**: Distinguishes real from fake samples

Trained adversarially: generator tries to fool discriminator, discriminator tries to detect fakes. Training challenges include mode collapse, non-convergence, vanishing gradients. Applications: image generation, style transfer, data augmentation, super-resolution.

Q46: What is reinforcement learning?

Answer: RL trains agents to make sequential decisions by interacting with an environment to maximize cumulative reward. Key concepts:

- State, action, reward, policy
- Value function: expected future rewards
- Q-learning: learn action-value function
- Policy gradient methods
- Exploration vs exploitation

Applications: game playing, robotics, autonomous vehicles, recommendation systems. Algorithms: Q-learning, DQN, A3C, PPO, DDPG.

Q47: Explain autoencoders and their applications.

Answer: Autoencoders are neural networks trained to reconstruct input through a bottleneck layer, learning compressed representations. Architecture: encoder (compresses) + decoder (reconstructs). Types:

- Vanilla autoencoders: dimensionality reduction
- Denoising: learns robust features
- Variational (VAE): generates new samples
- Sparse: forces sparse representations

Applications: dimensionality reduction, anomaly detection, image denoising, generative modeling.

Q48: What is federated learning?

Answer: Federated learning trains models across decentralized devices holding local data, without exchanging data samples. Each device trains locally and only model updates are aggregated centrally. Benefits: privacy preservation, reduced communication costs, compliance with regulations. Challenges: non-IID data, communication efficiency, security, systems heterogeneity. Applications: mobile keyboards, healthcare, finance.

Q49: Explain few-shot learning.

Answer: Few-shot learning trains models to learn from very few examples (1-shot, 5-shot learning). Approaches:

- Meta-learning (learning to learn): MAML, Prototypical Networks
- Transfer learning: fine-tune pre-trained models
- Metric learning: learn embedding space where similar classes cluster

Important in domains with limited labeled data: medical imaging, rare disease detection, robotics.

Q50: What is model interpretability and why is it important?

Answer: Model interpretability explains how models make decisions. Important for:

- Trust and adoption
- Debugging and improvement
- Regulatory compliance
- Fairness and bias detection
- Scientific understanding

Techniques:

- Feature importance: SHAP, LIME
- Attention visualization
- Saliency maps for images
- Partial dependence plots
- Rule extraction from trees
- Model-agnostic methods

11. Advanced Deep Learning Architectures

Q51: What is ResNet and why are skip connections important?

Answer: ResNet (Residual Network) introduces skip connections that add the input of a layer to its output: F(x) + x. This solves the degradation problem in very deep networks where adding layers paradoxically worsens performance. Skip connections:

- Enable gradient flow through the network (combat vanishing gradients)
- Allow learning identity mappings
- Enable training of networks with 100+ layers
- Create implicit ensemble of shallower networks The residual learning reformulates layers as learning residual functions with reference to layer inputs.

Q52: Explain Inception networks and why they use multiple filter sizes.

Answer: Inception modules apply multiple convolutional filters (1x1, 3x3, 5x5) and pooling in parallel at the same level, concatenating results. Benefits:

- Captures features at different scales simultaneously
- Network chooses relevant receptive field sizes
- 1x1 convolutions reduce computational cost (dimensionality reduction)
- Increases network width without excessive parameters
- More efficient than simply stacking layers Used in GoogLeNet/Inception v1-v4 architectures.

Q53: What are attention mechanisms beyond transformers?

Answer: Attention mechanisms assign importance weights to different parts of input:

- **Seq2Seq attention**: Decoder attends to all encoder hidden states
- **Self-attention**: Each position attends to all positions in same sequence
- **Spatial attention**: Attend to important image regions (visual attention)
- **Channel attention**: Weight importance of feature channels (SENet)
- **Cross-attention**: Query from one sequence, key/value from another

- **Local attention**: Attend to window rather than full sequence Applications beyond NLP: image captioning, visual question answering, video analysis.

Q54: Explain U-Net architecture and its applications.

Answer: U-Net is a symmetric encoder-decoder architecture with skip connections between corresponding encoder-decoder levels. Originally designed for biomedical image segmentation:

- **Encoder**: Contracting path captures context (convolution + pooling)
- **Decoder**: Expanding path enables precise localization (upsampling + convolution)
- **Skip connections**: Combine high-resolution features with upsampled features Applications: medical image segmentation, satellite imagery, image restoration. The architecture preserves spatial information while capturing semantic context.

Q55: What is Neural Architecture Search (NAS)?

Answer: NAS automates the design of neural network architectures using:

- **Search space**: Defines possible architectures (operations, connections)
- **Search strategy**: How to explore space (RL, evolution, gradient-based)
- **Performance estimation**: Evaluate candidate architectures

Approaches:

- Reinforcement learning (NASNet)
- Evolutionary algorithms
- Differentiable NAS (DARTS) - makes architecture search differentiable
- One-shot NAS - trains supernet once

Challenges: computational cost, transferability. Produces state-of-the-art architectures for specific tasks.

Q56: Explain knowledge distillation.

Answer: Knowledge distillation transfers knowledge from a large "teacher" model to a smaller "student" model. The student learns from:

- Soft targets (teacher's probability distributions) - contain more information than hard labels
- Intermediate representations
- Attention maps

Process: Student trained to match teacher's outputs using temperature-scaled softmax. Benefits:

- Model compression
- Faster inference
- Deploy on resource-constrained devices

- Sometimes students outperform training from scratch

Used in: model deployment, transfer learning, ensemble compression.

Q57: What are Siamese networks and triplet loss?

Answer: Siamese networks learn embeddings by training identical subnetworks on pairs/triplets:

- **Pair-based**: Learn if two inputs are similar (contrastive loss)
- **Triplet-based**: Anchor, positive (similar), negative (dissimilar). Triplet loss: L = max(d(a,p) - d(a,n) + margin, 0)

Forces network to learn embeddings where similar items are close, dissimilar items far. Applications:

- Face verification/recognition
- Signature verification
- One-shot learning
- Similarity search
- Duplicate detection

Q58: Explain Graph Neural Networks (GNNs).

Answer: GNNs operate on graph-structured data where entities (nodes) have relationships (edges). Key concepts:

- **Message passing**: Nodes aggregate information from neighbors
- **Node embeddings**: Learn representations considering graph structure
- **Graph convolution**: Aggregate neighbor features (GCN)

Types:

- GCN (Graph Convolutional Networks)
- GraphSAGE (sampling-based)
- GAT (Graph Attention Networks)
- GIN (Graph Isomorphism Networks)

Applications: social networks, molecule property prediction, recommendation systems, knowledge graphs, traffic prediction.

---

12. Natural Language Processing (Advanced)

Q59: Explain the difference between BERT, GPT, and T5.

Answer:

- **BERT**: Bidirectional encoder, masked language modeling, best for understanding tasks (classification, NER, QA)
- **GPT**: Unidirectional decoder, autoregressive generation, best for text generation
- **T5**: Encoder-decoder, frames all tasks as text-to-text, versatile for both understanding and generation

Architecture differences affect their strengths: BERT sees full context but can't generate; GPT generates naturally but only sees left context; T5 balances both through seq2seq architecture.

Q60: What is tokenization and what are different tokenization methods?

Answer: Tokenization splits text into units (tokens). Methods:

- **Word-level**: Split by spaces/punctuation. Issues: large vocabulary, OOV words
- **Character-level**: No OOV, but long sequences, loses semantic meaning
- **Subword**: Balance between word and character
    - **BPE** (Byte Pair Encoding): Merges frequent character pairs
    - **WordPiece**: Similar to BPE, used in BERT
    - **SentencePiece**: Language-agnostic, treats space as token
    - **Unigram**: Probabilistic subword segmentation

Subword methods handle OOV words, reduce vocabulary size, capture morphology.

Q61: Explain positional encoding in transformers.

Answer: Transformers process tokens in parallel, losing sequential information. Positional encoding adds position information:

- **Sinusoidal**: PE(pos,2i) = sin(pos/10000^(2i/d)), PE(pos,2i+1) = cos(pos/10000^(2i/d))
    - Deterministic, extrapolates to longer sequences
- **Learned**: Embedding layer for positions
    - More flexible, requires training

Positional encodings are added to input embeddings, allowing model to use order information. Alternatives: relative positional encoding, rotary embeddings (RoPE).

Q62: What is beam search in sequence generation?

Answer: Beam search is a heuristic search algorithm for generating sequences. Instead of greedy search (taking top prediction at each step), it maintains k hypotheses (beam width):

1. At each step, expand all k sequences
2. Score all candidates
3. Keep top k
4. Repeat until end token or max length

Trade-offs:

- k=1: greedy search (fast, suboptimal)
- Large k: better quality, slower, diminishing returns
- Can add length normalization to avoid bias toward short sequences

Used in: machine translation, image captioning, speech recognition.

Q63: Explain named entity recognition (NER) and approaches.

Answer: NER identifies and classifies entities (person, organization, location, date) in text. Approaches:

- **Rule-based**: Regex, gazetteers. Fast but low coverage.
- **Traditional ML**: CRF, HMM with hand-crafted features
- **Deep learning**:
    - BiLSTM-CRF: Captures bidirectional context + structured prediction
    - BERT-based: Fine-tune pre-trained models
    - Span-based: Predict entity spans directly

Challenges: ambiguity, nested entities, domain-specific entities, multilingual NER. Evaluation metrics: precision, recall, F1 at entity level (exact match).

Q64: What is the attention mechanism's computational complexity and how do you reduce it?

Answer: Standard self-attention has $O(n^2d)$ complexity where n is sequence length, d is dimension. For long sequences, this is prohibitive. Reduction techniques:

- **Sparse attention**: Attend to subset of positions (Longformer, BigBird)
- **Linformer**: Project keys and values to lower dimension - O(nd)
- **Performer**: Use kernel methods to approximate attention - O(nd)
- **Reformer**: LSH attention groups similar queries/keys - O(n log n)
- **Sliding window**: Local attention within fixed window

Trade-off between efficiency and model capacity.

Q65: Explain few-shot prompting and in-context learning.

Answer: Large language models can learn tasks from examples provided in the prompt without gradient updates:

- **Zero-shot**: Task described in natural language
- **Few-shot**: Include k examples in prompt
- **Chain-of-thought**: Include reasoning steps in examples

Mechanisms:

- Models "learn" from prompt context
- No parameter updates
- Performance scales with model size
- Prompt engineering becomes crucial

Advantages: no training data needed, rapid adaptation. Limitations: prompt length constraints, inconsistent performance.

---

13. Computer Vision (Advanced)

Q66: Explain object detection algorithms (R-CNN, YOLO, SSD).

Answer:

- **R-CNN family**: Two-stage detectors
    - R-CNN: Selective search for regions, CNN features, SVM classification
    - Fast R-CNN: ROI pooling, single-stage training
    - Faster R-CNN: Region Proposal Network (RPN)
- **YOLO** (You Only Look Once): Single-stage, divides image into grid, predicts bounding boxes and classes per cell. Fast but struggles with small objects.
- **SSD** (Single Shot Detector): Predicts boxes at multiple scales using feature maps from different layers. Balance of speed and accuracy.
- **Modern**: EfficientDet, DETR (transformer-based, no anchors)

Q67: What is semantic segmentation vs instance segmentation?

Answer:

- **Semantic segmentation**: Classify each pixel into categories. Doesn't distinguish between instances (all cars labeled as "car"). Methods: FCN, U-Net, DeepLab.
- **Instance segmentation**: Identify and delineate each object instance separately. Distinguishes car1 from car2. Methods: Mask R-CNN, YOLACT.
- **Panoptic segmentation**: Combines both - semantic for background, instance for objects.

Evaluation metrics: IoU, pixel accuracy, mean IoU, dice coefficient.

Q68: Explain data augmentation techniques for computer vision.

Answer: Techniques to artificially increase training data: **Basic transformations:**

- Geometric: rotation, flipping, cropping, scaling, translation
- Color: brightness, contrast, saturation, hue adjustment
- Noise injection

Advanced:

- **Mixup**: Blend two images and labels
- **CutMix**: Replace image region with another image's patch
- **AutoAugment**: Learn augmentation policies
- **RandAugment**: Random augmentation with magnitude control
- **Test-time augmentation**: Augment during inference, average predictions

Domain-specific:

- Medical: elastic deformations
- Document: perspective transforms

Benefits: improved generalization, robustness, addresses overfitting.

Q69: What is transfer learning in computer vision and when to use it?

Answer: Use pre-trained models (typically trained on ImageNet) as starting point:

Strategies:

- **Feature extraction**: Freeze early layers, train only final classifier. Use when: small dataset, similar domain
- **Fine-tuning**: Unfreeze some layers, train with low learning rate. Use when: moderate dataset, somewhat different domain
- **Full fine-tuning**: Train all layers. Use when: large dataset, different domain

Benefits:

- Faster convergence
- Better performance with less data
- Transfer of low-level features (edges, textures)

Considerations:

- Similarity between source and target tasks
- Dataset size
- Computational resources

Q70: Explain anchor boxes in object detection.

Answer: Anchor boxes (prior boxes) are predefined bounding boxes of various sizes and aspect ratios placed at regular positions in feature maps.

Purpose:

- Handle objects of different sizes and shapes
- Convert detection to classification + regression problem

- Each anchor predicts: objectness score, class probabilities, box offsets

Design considerations:

- Number of anchors per location (typically 3-9)
- Aspect ratios (e.g., 1:1, 1:2, 2:1)
- Scale factors
- Can be learned from data (K-means clustering on training boxes)

Limitations:

- Hand-designed heuristics
- Many negative anchors
- Modern approaches: anchor-free detection (FCOS, CenterNet)

Q71: What is image segmentation using conditional random fields (CRF)?

Answer: CRFs model dependencies between pixel labels, refining segmentation outputs:

Energy function minimizes:

- Unary potential: Individual pixel classification confidence
- Pairwise potential: Encourages label consistency between similar adjacent pixels

Benefits:

- Sharper boundaries
- Spatial consistency
- Post-processing for neural network predictions

Applications:

- Semantic segmentation refinement (DeepLab + CRF)
- Medical image segmentation
- Scene understanding

Limitation: Computationally expensive; faster approximations exist (dense CRF).

---

14. Recommender Systems

Q72: Explain matrix factorization in collaborative filtering.

Answer: Decompose user-item interaction matrix R (m×n) into user matrix U (m×k) and item matrix V (n×k): $R \approx UV^T$

Approach:

- Learn latent factors for users and items

- Minimize: $||R - UV^T||^2 + \lambda(||U||^2 + ||V||^2)$
- Techniques: SVD, ALS (Alternating Least Squares), gradient descent

Advantages:

- Handles sparsity
- Discovers latent features
- Scalable

Limitations:

- Cold start problem
- Doesn't incorporate side information
- Linear latent factor model

Extensions: SVD++, factorization machines, neural matrix factorization.

Q73: What is the cold start problem and how do you address it?

Answer: Cold start occurs with new users/items lacking interaction history.

Solutions:

For new users:

- Popularity-based recommendations
- Content-based filtering using demographics
- Ask for initial preferences
- Social network information

For new items:

- Promote to exploratory users
- Content-based features
- Hybrid approaches
- Active learning to select informative users

For new system:

- Expert-curated lists
- Transfer learning from similar domains
- Rapid data collection strategies

Q74: Explain the exploration-exploitation tradeoff in recommendations.

Answer: Balance between:

- **Exploitation**: Recommend known good items (maximize immediate reward)

- **Exploration**: Recommend uncertain items (learn user preferences, discover gems)

Approaches:

- **ε-greedy**: Exploit with probability 1-ε, explore randomly with ε
- **Thompson Sampling**: Sample from posterior distribution
- **Upper Confidence Bound (UCB)**: Recommend items with high uncertainty-adjusted value
- **Contextual bandits**: Consider context in exploration decisions

Benefits:

- Discover new user interests
- Overcome filter bubbles
- Adapt to changing preferences
- Improve long-term engagement

Q75: What are embedding-based recommendation systems?

Answer: Learn dense vector representations for users and items in shared space:

Approaches:

- **Two-tower models**: Separate encoders for users and items, learn to align
- **Deep crossing**: Cross features at different levels
- **Neural collaborative filtering**: Replace dot product with neural network
- **Graph embeddings**: Model user-item interactions as graph (node2vec, GraphSAGE)

Advantages:

- Capture complex patterns
- Incorporate rich features (text, images, metadata)
- Efficient retrieval (approximate nearest neighbors)
- Transfer learning

Applications: YouTube recommendations, Pinterest, Spotify.

Q76: Explain diversity and serendipity in recommendations.

Answer:

- **Diversity**: Recommendations should cover different categories/types. Metrics: intra-list diversity, coverage.
- **Serendipity**: Surprising yet relevant recommendations (not obvious). "Pleasantly unexpected."

Why important:

- Prevent filter bubbles
- User satisfaction
- Discovery and engagement
- Avoid boredom

Techniques:

- Post-processing reranking
- MMR (Maximal Marginal Relevance)
- Determinantal Point Processes (DPP)
- Multi-objective optimization
- Exploration strategies

Trade-off: Accuracy vs diversity/serendipity.

---

15. Time Series & Forecasting

Q77: Explain ARIMA models for time series.

Answer: ARIMA (AutoRegressive Integrated Moving Average) models time series as:

- **AR(p)**: Linear combination of p past values
- **I(d)**: Differencing d times to make stationary
- **MA(q)**: Linear combination of q past errors

ARIMA(p,d,q) requires:

- Stationarity (after differencing)
- ACF/PACF plots to determine p, q
- ADF test for stationarity

Extensions:

- SARIMA: Seasonal ARIMA
- ARIMAX: With exogenous variables
- VARIMA: Vector (multivariate) ARIMA

Limitations:

- Assumes linearity
- Struggles with long-term dependencies
- Manual parameter selection

Q78: What are Prophet and other modern forecasting methods?

Answer: Prophet (Facebook):

- Additive model: y(t) = trend + seasonality + holidays + error
- Automatic detection of changepoints
- Handles missing data and outliers
- Easy to use, interpretable
- Best for business time series with strong seasonality

Other methods:

- **LSTM/GRU**: Capture long-term dependencies
- **Temporal Convolutional Networks**: Efficient, parallelizable
- **Transformers**: Attention for time series (Informer, Temporal Fusion Transformer)
- **N-BEATS**: Neural basis expansion, interpretable
- **DeepAR**: Probabilistic forecasting with RNNs

Q79: How do you handle seasonality in time series?

Answer: Detection:

- Visual inspection (plots)
- ACF/PACF analysis
- Spectral analysis
- Decomposition (additive/multiplicative)

Modeling:

- **Statistical**: Seasonal ARIMA, exponential smoothing (Holt-Winters)
- **Feature engineering**: Cyclical encoding (sin/cos), seasonal indicators
- **Deep learning**: Seasonal period as input, multi-head attention
- **Deseasonalization**: Remove seasonal component, forecast residuals, add back

Multiple seasonalities:

- Daily + weekly + yearly patterns
- TBATS, Prophet handle multiple seasonal periods

Q80: Explain autocorrelation and partial autocorrelation.

Answer:

- **Autocorrelation Function (ACF)**: Correlation between series and its lagged values. Measures total correlation including indirect effects.
- **Partial Autocorrelation Function (PACF)**: Correlation between series and lag after removing effects of intermediate lags. Measures direct correlation only.

Usage:

- **ACF**: Identifies MA order (cut-off after q lags suggests MA(q))
- **PACF**: Identifies AR order (cut-off after p lags suggests AR(p))
- Together: Determine ARIMA parameters
- Check model adequacy (residual ACF should show no correlation)

Q81: What is stationarity and why is it important?

Answer: A time series is stationary if statistical properties (mean, variance, autocorrelation) don't change over time.

Types:

- **Strict stationarity**: Joint distribution identical across time shifts
- **Weak stationarity**: Constant mean, variance, and time-invariant autocovariance

Why important:

- Many models assume stationarity (ARIMA)
- Enables prediction (past statistical properties apply to future)
- Prevents spurious relationships

Tests:

- ADF (Augmented Dickey-Fuller)
- KPSS test
- Visual inspection

Achieving stationarity:

- Differencing
- Log transformation
- Detrending
- Seasonal adjustment

---

16. Anomaly Detection

Q82: Explain different approaches to anomaly detection.

Answer: Statistical methods:

- Z-score (standard deviations from mean)
- Grubb's test
- Boxplot (IQR method)

- Assumption: Data follows known distribution

Machine learning:

- **Isolation Forest**: Isolates anomalies in fewer splits
- **One-class SVM**: Learn boundary around normal data
- **LOF** (Local Outlier Factor): Density-based, local anomalies
- **DBSCAN**: Clustering-based, points not in clusters are anomalies

Deep learning:

- **Autoencoders**: High reconstruction error indicates anomaly
- **VAE**: Anomalies have low likelihood under learned distribution
- **GANs**: Discriminator identifies anomalies

Time series specific:

- ARIMA residuals
- Prophet anomaly detection
- LSTM prediction error

Q83: What is the difference between novelty and outlier detection?

Answer:

- **Outlier detection**: Training data contains outliers. Goal: Identify them among normal instances. Semi-supervised or unsupervised.
- **Novelty detection**: Training data is clean (no outliers). Goal: Detect if new observation is novel. One-class learning problem.

Methods:

- **Outlier**: Robust methods that tolerate contamination (Isolation Forest, LOF)
- **Novelty**: Learn boundary of normal (One-class SVM, Gaussian density estimation)

Evaluation challenges:

- Imbalanced data
- Labeling anomalies is expensive
- Metrics: precision-recall, F1, AUC-ROC
- Domain-specific definition of "anomaly"

Q84: How do autoencoders work for anomaly detection?

Answer: Autoencoders learn to compress and reconstruct normal data. Anomalies have high reconstruction error.

Architecture:

- **Encoder**: Compresses input to latent representation
- **Decoder**: Reconstructs from latent code
- **Loss**: Reconstruction error (MSE)

Process:

1. Train on normal data only
2. Anomalies reconstruct poorly (haven't learned their patterns)
3. Threshold reconstruction error

Variants:

- **Denoising**: Add noise, reconstruct clean version
- **Variational (VAE)**: Probabilistic, use likelihood for scoring
- **LSTM-based**: For sequential/time series data

Advantages: Unsupervised, learns complex patterns. **Challenge:** Threshold selection.

---

17. Ensemble Methods (Advanced)

Q85: Explain stacking and how it differs from bagging/boosting.

Answer: Stacking combines multiple models using a meta-learner:

Process:

1. Train base models on training data
2. Generate predictions on validation set
3. Train meta-model using base model predictions as features
4. Final prediction: Meta-model output

Differences:

- **Bagging/Boosting**: Combine same algorithm type, simple averaging/weighted voting
- **Stacking**: Combine diverse algorithms, learns optimal combination

Benefits:

- Leverages strengths of different algorithms
- Meta-learner learns when to trust each base model
- Often wins competitions

Challenges:

- Computational cost
- Risk of overfitting

- Complex pipeline

Q86: What is gradient boosting and how does XGBoost improve it?

Answer: Gradient Boosting:

- Sequential training: Each tree corrects previous trees' errors
- Fits new model to negative gradients of loss
- Final prediction: Weighted sum of all trees

XGBoost improvements:

- **Regularization**: L1/L2 on leaf weights, prevents overfitting
- **Sparsity awareness**: Handles missing values automatically
- **Weighted quantile sketch**: Efficient split finding
- **Cache-aware**: Optimized memory access
- **Parallel processing**: Tree construction, not sequential boosting
- **Tree pruning**: Max_depth first, then prune backward
- **Built-in CV**: Cross-validation in training

LightGBM/CatBoost: Further optimizations (histogram-based, categorical handling).

Q87: Explain the concept of out-of-bag error.

Answer: In bagging/Random Forest, each tree trains on bootstrapped sample (~63% of data). Remaining 37% are "out-of-bag" (OOB).

OOB Error:

- For each sample, predict using only trees where it was OOB
- Calculate error across all samples
- Provides unbiased error estimate without separate validation set

Benefits:

- Free validation (no data split needed)
- Reliable estimate similar to cross-validation
- Automatically computed during training
- Can monitor during training

Uses:

- Model selection
- Early stopping
- Feature importance calculation

Q88: What is the difference between hard and soft voting in ensembles?

Answer: Hard Voting:

- Each model votes for a class
- Final prediction: Majority vote
- Treats all models equally (or can weight votes)
- Simple, interpretable

Soft Voting:

- Each model outputs probability distribution
- Average probabilities across models
- Final prediction: Class with highest average probability
- Uses confidence information

When to use:

- **Hard**: Models don't output probabilities or probabilities are unreliable
- **Soft**: Generally better when probabilities are well-calibrated
- Soft voting usually outperforms hard voting

Weighted voting: Assign different weights based on model performance.

---

18. Model Interpretability & Explainability

Q89: Explain SHAP (SHapley Additive exPlanations).

Answer: SHAP assigns each feature an importance value for a prediction using game theory (Shapley values).

Key concepts:

- **Shapley value**: Fair distribution of "payout" among players
- Each feature is a "player," prediction is "payout"
- Considers all possible feature coalitions
- Satisfies desirable properties: local accuracy, missingness, consistency

Types:

- **TreeSHAP**: Fast for tree-based models
- **KernelSHAP**: Model-agnostic approximation
- **DeepSHAP**: For neural networks

Visualizations:

- Force plots (individual predictions)
- Summary plots (global feature importance)
- Dependence plots (feature interactions)

Advantages: Theoretically sound, consistent, both local and global explanations.

Q90: What is LIME and how does it work?

Answer: LIME (Local Interpretable Model-agnostic Explanations) explains individual predictions by approximating the model locally with an interpretable model.

Process:

1. Perturb input (generate similar samples)
2. Get predictions from black-box model
3. Weight samples by proximity to original
4. Train simple model (linear/tree) on perturbed data
5. Use simple model as explanation

Advantages:

- Model-agnostic
- Human-interpretable
- Flexible (text, images, tabular)

Limitations:

- Instability (different runs give different explanations)
- Choice of perturbation strategy affects results
- Local only (doesn't explain global behavior)

Use cases: Debugging, trust building, regulatory compliance.

Q91: How do you interpret feature importance in Random Forests?

Answer: Multiple methods:

1. Impurity-based (Gini importance):

- Total reduction in impurity when splitting on feature
- Fast, built-in
- Biased toward high-cardinality features

2. Permutation importance:

- Shuffle feature values, measure performance drop
- More reliable

- Computationally expensive

3. Drop-column importance:

- Retrain without feature, measure performance drop
- Most accurate, very expensive

Considerations:

- Correlation affects importance
- Check multiple methods
- Use SHAP for interactions
- Accumulated Local Effects (ALE) for dependencies

Q92: Explain attention visualization in neural networks.

Answer: Attention weights show which inputs the model focuses on:

In NLP:

- Visualize attention matrices (heatmaps)
- Shows which words model attends to
- Multi-head attention: Different heads capture different patterns
- Example: "it" refers to which noun?

In Vision:

- Attention maps overlay on images
- Highlights important regions
- Used in: image captioning, VQA, object detection
- GradCAM: Uses gradients to identify important regions

Interpretation cautions:

- High attention ≠ causal importance
- Attention is one component; doesn't explain full decision
- Can be manipulated without changing output
- Use alongside other explanation methods

Q93: What is model calibration and why does it matter?

Answer: A calibrated model's predicted probabilities match true frequencies:

- If model predicts 70% probability, should be correct 70% of time
- Calibration curve plots predicted vs actual probabilities

Why important:

- Medical diagnosis (trust probabilities)
- Risk assessment
- Decision-making under uncertainty
- Cost-sensitive applications

Calibration methods:

- **Platt scaling**: Fit logistic regression on validation predictions
- **Isotonic regression**: Non-parametric, more flexible
- **Temperature scaling**: Single parameter for neural networks
- **Beta calibration**: Parametric extension of Platt scaling

Metrics:

- Expected Calibration Error (ECE)
- Brier score
- Reliability diagrams

---

19. Optimization & Training Techniques

Q94: Explain different optimization algorithms (SGD, Adam, RMSprop).

Answer: SGD (Stochastic Gradient Descent):

- $\theta = \theta - \alpha \nabla J(\theta)$
- Simple, can escape local minima (noise)
- Requires careful learning rate tuning

SGD with Momentum:

- Accumulates velocity: $v = \beta v + \nabla J(\theta)$, $\theta = \theta - \alpha v$
- Accelerates in consistent directions, dampens oscillations
- β typically 0.9

RMSprop:

- Adapts learning rate per parameter
- Uses exponential moving average of squared gradients
- Good for non-stationary problems

Adam (Adaptive Moment Estimation):

- Combines momentum and RMSprop
- Maintains both first moment (mean) and second moment (variance)

- Most popular in deep learning
- Variants: AdamW (decoupled weight decay), AMSGrad

Choosing:

- Adam: Default choice, works well generally
- SGD+Momentum: Better generalization for vision tasks
- RMSprop: Good for RNNs

Q95: What is learning rate scheduling?

Answer: Adjust learning rate during training:

Strategies:

- **Step decay**: Reduce by factor every n epochs
- **Exponential decay**: $\alpha(t) = \alpha_0 e^{(-kt)}$
- **Cosine annealing**: Smooth decrease following cosine curve
- **Reduce on plateau**: Decrease when validation metric stagnates
- **Cyclical**: Oscillate between bounds (CLR)
- **Warmup**: Start small, increase to base rate, then decay
- **One-cycle**: Warmup, then cosine decay to very low

Benefits:

- Escape local minima early (high LR)
- Converge precisely later (low LR)
- Improve generalization

Implementation: Built into optimizers (PyTorch schedulers, TF callbacks).

Q96: Explain gradient clipping.

Answer: Limits gradient magnitude to prevent exploding gradients:

Methods:

- **Norm clipping**: If $||g|| > \text{threshold}$, scale to $g = g \times \text{threshold}/||g||$
- **Value clipping**: Clip each element to [-threshold, threshold]

When needed:

- RNNs/LSTMs (gradient flow through time)
- Very deep networks
- Reinforcement learning
- When gradients periodically explode

Typical values: Clip at 1.0 to 5.0 for gradient norm

Benefits:

- Training stability
- Prevents NaN/Inf
- Allows higher learning rates

Trade-off: May slow convergence if threshold too conservative.

Q97: What is mixed precision training?

Answer: Use lower precision (FP16) for most computations, higher precision (FP32) for critical operations:

Components:

- **FP16 computation**: Faster, less memory
- **FP32 master weights**: Prevent underflow in weight updates
- **Loss scaling**: Multiply loss to prevent gradient underflow
- **Dynamic loss scaling**: Automatically adjust scale factor

Benefits:

- 2-3× faster training
- 2× memory reduction (larger batches/models)
- Maintained accuracy with proper implementation

Requirements:

- Modern GPUs (Tensor Cores)
- Framework support (Apex, TF mixed precision)

Caution: Some operations (batch norm) need FP32.

Q98 Explain data parallelism vs model parallelism.

Answer: Data Parallelism:

- Replicate model across devices
- Split data batches across devices
- Each device computes gradients
- Synchronize gradients (all-reduce)
- Most common approach

Model Parallelism:

- Split model across devices

- Different devices hold different layers/parameters
- Forward/backward passes require communication
- Used when model doesn't fit on single device

Hybrid: Combine both (large models like GPT-3)

Pipeline Parallelism: Model parallelism with micro-batches to improve utilization

Considerations:

- Communication overhead
- Load balancing
- Gradient synchronization strategy

---

20. ML System Design & Production (Advanced)

Q99: How do you handle data versioning and experiment tracking?

Answer: Data Versioning:

- **DVC** (Data Version Control): Git-like for data
- **LakeFS**: Git-like for data lakes
- **Delta Lake**: ACID transactions for data lakes
- Store data snapshots with metadata
- Track data lineage and transformations

Experiment Tracking:

- **MLflow**: Track parameters, metrics, artifacts
- **Weights & Biases**: Visualization, collaboration
- **Neptune.ai**: ML metadata store
- **TensorBoard**: Visualize training

Track:

- Hyperparameters
- Metrics (train/val/test)
- Model artifacts
- Environment (dependencies, hardware)
- Code version (git commit)
- Data version

Benefits: Reproducibility, collaboration, audit trail, debugging.

Q100: Explain feature stores and why they're important.

Answer: Centralized repository for storing and serving features:

Components:

- **Offline store**: Historical features for training (batch)
- **Online store**: Low-latency serving for inference (real-time)
- **Feature registry**: Metadata, documentation
- **Feature computation**: Transform raw data to features

Benefits:

- **Consistency**: Same features for training and serving
- **Reusability**: Share features across teams
- **Freshness**: Automatic feature updates
- **Discovery**: Catalog of available features
- **Monitoring**: Track feature drift

Examples: Feast, Tecton, AWS SageMaker Feature Store, Databricks Feature Store

Use cases: Recommendation systems, fraud detection, personalization.

Q101: How do you design a real-time ML system?

Answer: Architecture components:

- **Stream processing**: Kafka, Kinesis for real-time data
- **Feature computation**: Flink, Spark Streaming
- **Model serving**: TensorFlow Serving, TorchServe, custom REST API
- **Caching**: Redis for frequent predictions
- **Load balancing**: Distribute traffic
- **Monitoring**: Latency, throughput, errors

Design considerations:

- **Latency requirements**: p50, p95, p99 targets
- **Throughput**: QPS (queries per second)
- **Model complexity vs speed**: Model size, inference time
- **Batching**: Batch requests for efficiency
- **Fallback**: Simpler model if primary fails
- **A/B testing**: Gradual rollout

Optimizations:

- Model quantization
- TensorRT, ONNX Runtime
- GPU inference
- Model distillation

Q102: Explain online learning and when to use it.

Answer: Models continuously learn from new data:

Approaches:

- **Incremental learning**: Update model with new data batches
- **Stream learning**: Update on individual samples
- **Mini-batch online learning**: Update on small batches

Algorithms:

- Online gradient descent
- Vowpal Wabbit
- River (formerly creme)
- Online random forests
- Incremental PCA

When to use:

- Data distribution changes rapidly
- Concept drift
- Large-scale data (doesn't fit in memory)
- Need immediate adaptation
- Examples: Ad click prediction, recommendation systems

Challenges:

- Catastrophic forgetting
- Stability vs plasticity
- Validation strategy
- Computational constraints

Q103: How do you handle model versioning and rollback?

Answer: Versioning:

- Semantic versioning (major.minor.patch)
- Tag models with metadata (date, performance, data version)

- Store in model registry (MLflow, TensorFlow Hub)
- Track lineage (training data, code, parameters)

Deployment strategies:

- **Blue-green**: Two environments, instant switch
- **Canary**: Gradual traffic shift (5% → 50% → 100%)
- **Shadow mode**: New model runs but doesn't serve; compare outputs
- **A/B testing**: Split traffic, compare metrics

Rollback:

- Keep previous versions deployed
- Automated rollback triggers (performance drops, error spikes)
- Gradual rollback (reverse canary)
- Feature flags for instant toggle

Monitoring:

- Performance degradation detection
- Comparison with baseline
- Business metrics impact

Q104: Explain the concept of technical debt in ML systems.

Answer: Accumulation of shortcuts and suboptimal decisions:

Types:

- **Data debt**: Pipeline brittle, dependencies unclear
- **Model debt**: Glue code, pipeline jungles
- **Configuration debt**: Difficult to manage configurations
- **Monitoring debt**: Insufficient observability
- **Infrastructure debt**: Tightly coupled systems
- **Testing debt**: Inadequate validation

Examples:

- Dead experimental code paths
- Undocumented data dependencies
- Configuration by copy-paste
- Manual feature engineering
- Ad-hoc monitoring

Solutions:

- Automated testing (data, model, infrastructure)
- Clear abstractions and interfaces
- Documentation and versioning
- Refactoring sprints
- Technical debt tracking

Consequences: Slower iteration, bugs, maintenance burden, difficulty scaling.

Q105: How do you design for fairness in ML systems?

Answer: Fairness metrics:

- **Demographic parity**: Equal positive rate across groups
- **Equalized odds**: Equal TPR and FPR across groups
- **Predictive parity**: Equal PPV across groups
- **Individual fairness**: Similar individuals get similar predictions

Techniques:

- **Pre-processing**: Balance training data, reweighting
- **In-processing**: Fairness constraints during training
- **Post-processing**: Adjust predictions to meet fairness criteria

Process:

- Define protected attributes (race, gender, age)
- Choose fairness metric (context-dependent)
- Measure bias in data and predictions
- Implement interventions
- Monitor in production

Trade-offs:

- Accuracy vs fairness
- Different fairness definitions can conflict
- Legal and ethical considerations

Tools: Fairlearn, AI Fairness 360, What-If Tool.

---

21. Advanced Topics & Emerging Areas

Q106: Explain contrastive learning.

Answer: Learn representations by comparing similar (positive) and dissimilar (negative) pairs:

Key concepts:

- Positive pairs: Same image with augmentations, or related items
- Negative pairs: Different images/items
- Loss: Pull positives together, push negatives apart

Algorithms:

- **SimCLR**: Contrastive learning for images
- **MoCo** (Momentum Contrast): Queue for negatives
- **CLIP**: Contrastive text-image pairs
- **SwAV**: Clustering-based

Benefits:

- Self-supervised learning (no labels needed)
- Strong representations for downstream tasks
- Works with limited labeled data

Applications: Pre-training for transfer learning, few-shot learning, retrieval.

Q107: What is self-supervised learning?

Answer: Learning from unlabeled data by creating pretext tasks:

Approaches:

Vision:

- Colorization: Predict color from grayscale
- Rotation prediction: Predict image rotation
- Jigsaw puzzles: Arrange shuffled patches
- Contrastive learning (SimCLR, MoCo)
- Masked image modeling (MAE)

NLP:

- Masked language modeling (BERT)
- Next sentence prediction
- Autoregressive modeling (GPT)

Benefits:

- Leverage massive unlabeled data
- Pre-training for downstream tasks

- Reduces annotation cost

Intuition: Create supervisory signal from data structure itself.

Q108: Explain neural ordinary differential equations (Neural ODEs).

Answer: Model hidden layer transformations as continuous ordinary differential equations:

Standard NN: $h_{t+1} = h_t + f(h_t, \theta)$ **Neural ODE:** $dh(t)/dt = f(h(t), t, \theta)$, solve with ODE solver

Benefits:

- Constant memory cost (regardless of depth)
- Adaptive computation (ODE solver adjusts step size)
- Continuous-depth models
- Better for irregular time series

Applications:

- Time series modeling
- Normalizing flows for density estimation
- Continuous normalizing flows
- Physics-informed models

Challenges: Slower training, ODE solver stability.

Q109: What is meta-learning (learning to learn)?

Answer: Learn algorithms/initialization that quickly adapt to new tasks:

Approaches:

Model-based:

- Learn model that takes tasks as input, outputs predictions
- Memory-augmented networks

Metric-based:

- Learn embedding space where similar instances cluster
- Prototypical Networks, Matching Networks, Siamese Networks

Optimization-based:

- MAML (Model-Agnostic Meta-Learning): Learn initialization that few-shot fine-tunes well
- Reptile: First-order MAML approximation

Applications:

- Few-shot learning
- Domain adaptation

- Hyperparameter optimization
- Neural architecture search

Goal: Quickly adapt to new tasks with few examples.

Q110: Explain causal inference in machine learning.

Answer: Understanding cause-and-effect relationships, not just correlations:

Key concepts:

- **Confounders**: Variables affecting both treatment and outcome
- **Counterfactuals**: What would have happened without treatment?
- **Treatment effect**: Causal impact of intervention

Methods:

- **Randomized experiments**: A/B tests (gold standard)
- **Propensity score matching**: Match treated/control units
- **Instrumental variables**: Use variable affecting treatment but not outcome directly
- **Difference-in-differences**: Compare changes over time
- **Causal graphs (DAGs)**: Represent causal relationships

ML approaches:

- Causal forests
- Double machine learning
- Neural causal models
- Counterfactual inference with deep learning

Applications: Policy decisions, personalization, understanding model behavior.

Q111: What is continual/lifelong learning?

Answer: Learning multiple tasks sequentially without forgetting previous ones:

Challenge: Catastrophic forgetting - new task overwrites old knowledge

Approaches:

Regularization-based:

- **EWC** (Elastic Weight Consolidation): Penalize changes to important weights
- **SI** (Synaptic Intelligence): Track parameter importance online

Replay-based:

- Store subset of old data, interleave with new
- Generative replay: Generate pseudo-samples from old tasks

Architecture-based:

- **Progressive networks**: Add new modules for new tasks
- **PackNet**: Prune network, freeze, use remaining for new task

Dynamic architectures:

- Grow network for new tasks
- Learn task routing

Applications: Robotics, personalized systems, edge devices with limited data.